

วิเคราะห์เชิงลึก Google AI Edge Gallery repository

Google AI Edge Gallery เป็นแอป on-device LLM แบบ native (Kotlin + Swift) ที่ขับเคลื่อนด้วย LiteRT-LM (Google AI) (github) ซึ่งเป็น inference runtime ของ Google เอง ไม่ได้ใช้ llama.cpp Repository นี้รองรับทั้ง Android และ iOS (Hugging Face +2) โดยเน้นโมเดลตระกูล Gemma เป็นหลัก พร้อมรองรับ Llama, Phi-4, Qwen (GitHub) (GitHub) และ DeepSeek ด้วย แอปทั้งหมดทำงาน offline บนอุปกรณ์ 100% โดยไม่ส่งข้อมูลขึ้น cloud (GitHub) (AIQnAHUB)

1. MediaPipe ถูกใช้อย่างกว้างขวาง แต่กำลังเปลี่ยนผ่านไป LiteRT-LM

ไฟล์ (libs.versions.toml) ของโปรเจกต์ระบุ dependency ของ MediaPipe ชัดเจน 3 ตัว คือ (com.google.mediapipe:tasks-genai) (v0.10.27) สำหรับ LLM Inference API (GitHub) ซึ่งเป็นตัวขับเคลื่อนหลักของการรัน LLM บนอุปกรณ์, (com.google.mediapipe:tasks-text) (v0.10.26.1) (GitHub) สำหรับงาน NLP เช่น text embedding, และ (com.google.mediapipe:tasks-vision-image-generator) (v0.10.26.1) (GitHub) สำหรับฟีเจอร์ image generation

ข้อสำคัญคือ Gallery v1.0.8 (ธันวาคม 2025) ได้ migrate core LLM inference ไปใช้ LiteRT-LM Kotlin API แล้ว Release notes ระบุว่า "Migrated to the LiteRT-LM Kotlin API" (GitHub) ดังนั้น ปัจจุบันตัว text-based LLM inference ใช้ LiteRT-LM เป็นหลัก แต่ MediaPipe dependencies ยังคงอยู่ใน build file สำหรับงานเฉพาะทาง เช่น audio understanding (Speech-to-Text ผ่าน Gemma 3n) ที่ยังทำงานผ่าน MediaPipe LLM Inference API อยู่ (Google Developers) ทั้งนี้ Google ได้ประกาศ deprecate MediaPipe LLM Inference API อย่างเป็นทางการแล้ว และแนะนำให้ migrate ไป LiteRT-LM (Google AI) (Google AI)

2. ไม่ใช่ llama.cpp — ใช้ LiteRT-LM ที่เป็น engine ของ Google เอง

Gallery ไม่ได้ใช้ llama.cpp เลย ทั้งใน build.gradle, source code และ dependency tree ไม่มีร่องรอยของ llama.cpp, GGML หรือ GGUF format ใดๆ Inference engine ที่ใช้คือ LiteRT-LM ซึ่ง Google พัฒนาเอง (Google AI +2) สร้างอยู่บน LiteRT (ตัวต่อยอดจาก TensorFlow Lite) (github +2) สถาปัตยกรรมจากล่างขึ้นบนเป็นดังนี้:

- XNNPACK → optimized CPU operators สำหรับ ARM/x86 (GitHub)
- ML Drift → GPU engine รองรับ OpenCL, OpenGL, Metal, WebGPU (Google Developers)
- LiteRT → core runtime จัดการ model loading, hardware acceleration (Google AI)
- LiteRT-LM → orchestration layer สำหรับ LLM โดยเฉพาะ (Hugging Face) จัดการ tokenization, KV-cache, conversation sessions, tool use (Google Developers)

Google เอง benchmark LiteRT-LM เทียบกับ llama.cpp อย่างชัดเจน โดยอ้างว่า LiteRT เร็วกว่าทั้ง prefill และ decode บน Samsung Galaxy S25 Ultra (Google Developers) ซึ่งยืนยันว่าทั้งสองเป็น competing runtime ที่แยกจากกันโดยสิ้นเชิง

โมเดลใช้ format `.litertlm` `(github)` (ไม่ใช่ `.gguf`) ของ llama.cpp) และยังรองรับ `(.task)` (MediaPipe legacy) กับ `(.tflite)` `(Google AI)` ด้วย

3. โมเดลที่รองรับครอบคลุมตั้งแต่ Gemma ไปจนถึง DeepSeek

Gallery รองรับ open-weight models หลายตระกูล `(Hugging Face)` โดยดาวน์โหลดจาก Hugging Face โดยตรง: `(github)`

โมเดล	Quantization	Context Length	ขนาดไฟล์
Gemma 4 E2B / E4B	2-bit, 4-bit	128K	~1.5-4 GB
Gemma 3n E2B	4-bit	4096	~2.9 GB
Gemma 3n E4B	4-bit	4096	~4.2 GB
Gemma 3 1B	4-bit	4096	~557 MB
FunctionGemma 270M	8-bit	1024	~288 MB
Llama 3.2 1B Instruct	8-bit	8192	~1.1 GB
Llama 3.2 3B Instruct	8-bit	8192	~2.9 GB
Phi-4-mini	8-bit	4096	~3.7 GB
Qwen 2.5 1.5B	8-bit	4096	~1.5 GB
DeepSeek-R1-Distill-Qwen-1.5B	—	—	—

Gemma 4 คือโมเดลเด่นล่าสุด `(google)` ที่เพิ่งเปิดตัว `(App Store)` รองรับ Thinking Mode ที่แสดงกระบวนการคิดแบบ step-by-step, `(GitHub)` `(App Store)` multimodal (text + image + audio), และ function calling `(GitHub)` ที่สำคัญคือ **ไม่ได้ใช้ Gemini Nano** — โมเดลทั้งหมดเป็น open-weight ที่ดาวน์โหลดจาก Hugging Face ได้อย่างอิสระ `(Hugging Face)` `(GitHub)` ต่างจาก Gemini Nano ที่เป็น proprietary model ผูกกับ Google AI Core

4. พัฒนาด้วย Native ทั้งหมด — ไม่ใช่ Flutter

Gallery **ไม่ได้ใช้ Flutter** แอปพัฒนาแบบ native ทั้งสอง platform:

- **Android:** เขียนด้วย Kotlin (99.8% ของ codebase ใน repo) ใช้ Jetpack Compose `(github)` สำหรับ UI
- **iOS:** เขียนด้วย Swift `(Medium)` (อ้างอิงจากบล็อกและ App Store) `(Medium)` น่าจะใช้ SwiftUI (เนื่องจาก minimum iOS 17) `(App Store)` `(GitHub)`

ไม่มี `pubspec.yaml`, ไม่มี Dart code, ไม่มี Flutter dependency ใดๆ ใน repository การเลือก native มีเหตุผลชัดเจนคือต้องเข้าถึง hardware acceleration (GPU/NPU) โดยตรงผ่าน low-level API ซึ่ง Flutter จะเป็น overhead ที่ไม่จำเป็น

5. Tech stack ที่ประกอบด้วยเครื่องมือ production-grade ของ Google

Android tech stack ทั้งหมด ที่ยืนยันจาก `build.gradle.kts` และ `libs.versions.toml`:

Inference & ML: LiteRT-LM (v0.0.0-alpha06+), MediaPipe tasks-genai/text/vision, Play Services TFLite Java & GPU (v16.4.0) (`deepwiki`) — ครอบคลุมทั้ง LLM inference, text processing และ image generation พร้อม GPU acceleration ผ่าน OpenCL

UI & Architecture: Jetpack Compose (BOM 2025.11.01), Hilt (v2.57) สำหรับ dependency injection, Navigation Compose (v2.8.9), (`deepwiki`) Material Icons Extended, SplashScreen API — สถาปัตยกรรมแบบ MVVM ตาม modern Android best practices

Data & Networking: Protocol Buffers + DataStore สำหรับ type-safe persistence ของ model metadata, (`DeepWiki`) Kotlin Serialization + Gson สำหรับ JSON, AppAuth (v0.11.1) สำหรับ **HuggingFace OAuth** authentication, (`DeepWiki`) (`GitHub`) WorkManager สำหรับ background model download

Media & Camera: CameraX (v1.4.2) สำหรับฟีเจอร์ Ask Image, Exifinterface สำหรับ image metadata

Services: Firebase (BOM v33.16+), Google Services, OSS Licenses สำหรับ license compliance, Commonmark + Richtext สำหรับ Markdown rendering ใน chat UI, Security Crypto สำหรับ encrypted storage

Build: AGP 8.8.2+, Kotlin 2.1.0+, compileSdk 35 (Android 15), minSdk 31 (Android 12), Java 11 compatibility, (`deepwiki`) Protobuf plugin สำหรับ code generation

6. รองรับทั้ง Android และ iOS แต่ source code ของ iOS ยังไม่เปิดเผย

Gallery รองรับทั้งสอง platform — Android 12+ และ iOS 17+ (`GitHub`) มีแอปบน **Google Play Store** และ **Apple App Store** (`GitHub`) พร้อมฟีเจอร์เทียบเคียงกัน (AI Chat, Ask Image, Audio Scribe, Prompt Lab, Agent Skills, Mobile Actions, Tiny Garden) (`GitHub`)

อย่างไรก็ตาม มีจุดที่น่าสนใจ: **source code ของ iOS ยังไม่อยู่ใน public repository** โครงสร้างโฟลเดอร์มีเพียง `Android/` เท่านั้น ไม่พบ `iOS/` folder, ไม่มี Package.swift, ไม่มี Podfile `GitHub` แสดงภาษาเป็น Kotlin 99.8% (`github`) ทั้งที่บทความจาก Google DevRel อ้างว่า "The app is entirely open-source (Kotlin on Android, Swift on iOS)" (`Medium`) สิ่งนี้อาจหมายความว่า iOS code อยู่ใน internal repository หรือยังไม่ได้เผยแพร่ตามที่ประกาศ

บน iOS, แอปน่าจะใช้ **LiteRT-LM C++ core ที่ bridge ไปหา Swift** เนื่องจาก native Swift SDK ของ LiteRT-LM ยังอยู่ในขั้น "Coming Soon" (`github +2`) นักพัฒนานักคลที่สามรายงานว่ายังไม่สามารถ build

GPU accelerator สำหรับ iOS ได้ง่ายขึ้น ขณะที่แอป Gallery ของ Google ทำได้ — แสดงว่า Google ใช้ internal build system (Bazel) กับ pre-built frameworks (GitHub)

7. LiteRT เป็นแกนหลัก — ไม่ใช่ ExecuTorch

ใช้ **LiteRT (TensorFlow Lite successor)** อย่างเต็มรูปแบบ (GitHub) ทั้งผ่าน LiteRT-LM โดยตรง และผ่าน (play-services-tflite-java) + (play-services-tflite-gpu) (v16.4.0) ที่ให้ GPU-accelerated inference ผ่าน Google Play Services

ไม่ได้ใช้ ExecuTorch ซึ่งเป็น on-device inference framework ของ Meta/PyTorch Gallery เป็นโปรเจกต์ของ Google ที่ใช้ Google stack ทั้งหมด (LiteRT → LiteRT-LM → MediaPipe) ไม่มี ExecuTorch dependency ใดๆ ในโปรเจกต์

AndroidManifest.xml ยังระบุ native libraries ที่เกี่ยวข้องกับ hardware acceleration ได้แก่ (libOpenCL.so) (สำหรับ GPU inference) และ (libvndksupport.so) (vendor NDK support) ทั้งคู่ยังเป็น optional เพื่อให้แอปทำงานได้แม้อุปกรณ์ไม่รองรับ GPU โดย fallback ไปใช้ CPU (DeepWiki)

สรุปภาพรวมทั้ง 7 ประเด็น

ประเด็น	คำตอบ
MediaPipe	✅ ใช้ tasks-genai (LLM), tasks-text, tasks-vision แต่กำลัง migrate ไป LiteRT-LM
llama.cpp	❌ ไม่ใช่ — ใช้ LiteRT-LM ของ Google แทน
โมเดล LLM	Gemma 4/3n/3 1B, FunctionGemma, Llama 3.2, Phi-4-mini, Qwen 2.5, DeepSeek-R1
Framework	Native — Kotlin (Android) + Swift (iOS) ไม่ใช่ Flutter
Tech stack	LiteRT-LM, Jetpack Compose, Hilt, CameraX, HuggingFace OAuth, Firebase
Platform	ทั้ง Android 12+ และ iOS 17+ (แต่ iOS source ยังไม่เปิดเผย)
LiteRT / ExecuTorch	✅ LiteRT เต็มรูปแบบ / ❌ ไม่ใช่ ExecuTorch